NEXUS Language Manual

Version 2.0.0
Author: fanxcz
Repository: https://github.com/fanxcz/nexus-lang

---

1. Introduction

NEXUS is a compiled programming language designed for native applications, tools, and games. The reference compiler is written in C++20. Programs are compiled to native code through LLVM IR or through a small built-in native backend.

The language is intentionally small, explicit, and predictable. It does not hide control flow or memory operations. Variables are stack slots, function calls are direct, and the generated LLVM IR can be inspected and compiled with a normal C++ toolchain.

NEXUS has three major parts:

1. **Compiler** (src/) – lexer, parser, semantic analysis, and code generation.
2. **Runtime** (src/runtime/runtime.cpp) – console, files, networking, graphics, audio, input, and helper functions.
3. **Standard library** (stdlib/) – convenience wrappers written in NEXUS.
4. **Self-hosting toolchain** (selfhost/) – lexer, parser, type checker, and code generator written in NEXUS.

The LLVM backend is the recommended backend. It supports the full language. The native backend

supports only a small subset and exists for experiments and tiny programs.

---

2. Compilation model

A NEXUS program is compiled in several stages:

1. **Lexical analysis** converts source text into tokens.
2. **Parsing** builds an abstract syntax tree.
3. **Semantic analysis** checks types and variable usage.
4. **Code generation** emits LLVM IR or native code.
5. **External compilation** (for LLVM) compiles .ll files with a clang-compatible driver and links the runtime.

This separation makes the compiler easier to debug. The generated .ll file is ordinary text and can be inspected directly.

---

3. Installation and build

3.1 Building from source

```
cmake -S . -B build
cmake --build build -j
ctest --test-dir build
```

The compiler is produced as build/nexus (or build/nexus.exe on Windows).

3.2 Environment variables

- NEXUS_CXX – clang-compatible driver used by self-hosting tests and by generated program compilation.

- NEXUS_SOURCE_ROOT — source root used by the compiler (set by CMake).

3.3 Self-hosting environment

For self-hosting tests on Windows, NEXUS_CXX can point to build/clang++.cmd, which wraps the bundled Zig toolchain.

---

4. First program

Create hello.nx:

```
fn main() {
    print("Hello, NEXUS!")
}
```

Run it:

```
nexus run hello.nx --backend llvm
```

What happens:

1. The compiler reads hello.nx.
2. It generates LLVM IR.
3. The runtime is linked.
4. The resulting program prints the message.

---

5. Lexical structure

5.1 Comments

Line comments begin with // and continue to the end of the line:

```
let x = 10 // comment
```

Block comments use /* and */:

```
/* This is
   a block comment. */
```

5.2 Identifiers

Identifiers consist of letters, digits, and underscores. They must not begin with a digit. Identifiers are case-sensitive.

5.3 Keywords

Keywords are reserved and cannot be used as identifiers:

```
fn let const mut return if else while for in break
struct enum import extern unsafe as match true fals
```

5.4 Literals

Integer literals are written in decimal form:

```
0
42
-7
```

Floating point literals contain a decimal point:

```
3.14
1.0
0.5
```

String literals are enclosed in double quotes:

```
"hello"
```

Boolean literals are:

```
true
false
```

---

6. Types

NEXUS has a small, explicit type system.

Type	Meaning
i64	signed 64-bit integer
f64	64-bit floating point number
bool	boolean value
string	text
void	no value

Types are not inferred for function parameters. Function signatures must state parameter and return types explicitly.

Type annotations are part of the function declaration:

```
fn scale(x: f64, factor: f64) -> f64 {
    return x * factor
}
```

---

7. Variables

7.1 Immutable variables

Use let to declare a variable whose value does not change:

```
let x = 10
```

After declaration, x cannot be reassigned.

7.2 Mutable variables

Use let mut when the value will change:

```
let mut i = 0
i = i + 1
```

7.3 Discarding values

Use let _ to explicitly discard a value:

```
let _ = some_function()
```

This is useful when a function returns a value that is not needed.

7.4 Type stability

A variable keeps its type for its entire lifetime. Reassigning a value of a different type is an error:

```
let x = 10
x = 20      // ok
x = "text"  // error
```

7.5 Scope

Variables are scoped to the block in which they are declared. A variable declared inside an if or while block is not visible outside that block.

---

8. Expressions

8.1 Arithmetic

```
a + b
a - b
a * b
a / b
a % b
```

Integer operations use signed 64-bit arithmetic. If either operand is f64, the operation is performed in floating point.

8.2 Comparisons

```
a == b
a ≠ b
a < b
a ≤ b
a > b
a ≥ b
```

Comparisons produce a bool.

8.3 Logical operators

```
a && b
a || b
!a
```

&& and || operate on boolean values.

8.4 Unary minus

```
-x
```

For i64, this computes zero minus the value. For f64, it negates the floating point value.

8.5 String concatenation

```
let s = "hello" + " " + "world"
```

The + operator concatenates strings when at least one operand is a string.

8.6 Precedence

From highest to lowest:

1. Primary expressions and function calls
2. Unary - and !
3. *, /, %
4. +, -
5. Comparisons
6. &&
7. ||

Parentheses can be used to override precedence.

---

9. Control flow

9.1 If statement

```
if condition {
    ...
} else {
    ...
}
```

The condition must be boolean. The else branch is optional.

9.2 While loop

```
while condition {
    ...
}
```

The condition is evaluated before each iteration.

9.3 Blocks

Blocks are delimited by braces. They introduce a new scope:

```
{
    let x = 1
    print(x)
}
```

---

10. Functions

10.1 Declaration

```
fn name(param: type, ...) → type {
    ...
}
```

10.2 Parameters

Parameters are named and typed:

```
fn add(a: i64, b: i64) → i64 {
    return a + b
}
```

10.3 Return

```
return value
```

A function may return early. If a function does not explicitly return, the compiler may insert a default return where supported.

10.4 Main function

```
fn main() {
    print("Hello")
}
```

main is the program entry point. At the LLVM level it returns i32.

10.5 Function calls

```
let x = add(1, 2)
print(add(x, 3))
```

10.6 Recursion

Functions may call themselves:

```
fn factorial(n: i64) → i64 {
    if n ≤ 1 {
        return 1
    }
    return n * factorial(n - 1)
}
```

10.7 Function order

Functions must be declared before they are called. The compiler processes declarations in source order.

---

11. Strings

11.1 Literals

```
let s = "hello"
```

11.2 Escape sequences

```
"\n"   // newline
"\t"   // tab
"\r"   // carriage return
"\\"   // backslash
"\""   // double quote
```

11.3 Concatenation

```
let s = "a" + " " + "b"
```

11.4 Printing strings

```
print("hello")
```

11.5 Byte-oriented operations

String operations in the standard library are byte-oriented. This keeps the implementation simple and predictable.

---

12. Floating point

Floating point values use the f64 type:

```
let x = 1.5
let y = 2.0
print(x + y)
print(x * y)
print(x / y)
```

Comparisons:

```
if x < y {
    print("less")
}
```

Floating point arithmetic follows IEEE-754 semantics through LLVM.

---

13. Modules and imports

Import a module by file name:

```
import "io.nx"
```

The standard library is located in stdlib/. Imports make the module's functions available.

---

14. Standard library

14.1 io

```
read_line(prompt: string) → string
read_i64(prompt: string) → i64
read_f64(prompt: string) → f64
```

14.2 fs

```
fs_read(path: string) → string
fs_write(path: string, data: string) → i64
fs_exists(path: string) → bool
```

14.3 string

```
len(s: string) → i64
byte_at(s: string, index: i64) → i64
byte_len(s: string) → i64
```

14.4 os

```
args_count() → i64
args_get(index: i64) → string
```

14.5 net

```
http_get_text(url: string) → string
```

14.6 json

```
json_field(json: string, key: string) → string
```

14.7 db

```
db_open(path: string) → i64
db_exec(db: i64, sql: string) → i64
db_close(db: i64)
```

14.8 crypto

```
hash_sha256(text: string) → string
```

14.9 memory

```
memory_alloc(bytes: i64) → i64
memory_free(ptr: i64)
```

---

15. Runtime functions

The runtime provides low-level functions used by generated code:

```
nexus_print_i64(i64)
nexus_print_f64(double)
nexus_print_bool(i1)
nexus_print_str(ptr)

nexus_str_concat(ptr, ptr) → ptr
nexus_str_equal(ptr, ptr) → i1

nexus_file_read(ptr) → ptr
nexus_file_write(ptr, ptr) → i64
nexus_file_exists(ptr) → i1

nexus_env(ptr) → ptr
nexus_system(ptr) → i64
nexus_exit(i64)
```

These functions are declared automatically in generated LLVM IR.

---

16. Tooling

16.1 Running programs

```
nexus run file.nx --backend llvm
```

16.2 Type checking

```
nexus check file.nx
```

16.3 Building

```
nexus build
```

16.4 REPL

```
nexus repl
```

16.5 Formatting

```
nexus fmt file.nx
```

16.6 Self-test

```
nexus self-test
```

16.7 Doctor

```
nexus doctor
```

16.8 Backends

- llvm supports the full language.
- native supports only a small subset.

Use --backend llvm for all real programs.

---

17. Build system

NEXUS uses CMake:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build
```

The build produces the compiler, tests, and runtime tests.

---

18. Native backend

The native backend is intentionally small. It is useful for experiments but does not support strings, functions with full type support, or most standard library features.

For any non-trivial program, use:

```
nexus run file.nx --backend llvm
```

---

19. Self-hosting

The selfhost/ directory contains a compiler written in NEXUS:

- lexer_lib.nx
- parser_lib.nx
- typecheck_lib.nx

- codegen_lib.nx

The self-hosted code generator supports:

- let and let mut
- assignment
- print
- arithmetic
- comparisons
- logical operators
- unary minus
- if and else
- while
- functions and calls
- string literals
- f64

Self-hosting is considered complete only when the self-hosted compiler produces output matching the C++ compiler on the test corpus.

---

20. Grammar

```
program      := declaration*
declaration  := function | statement
function     := "fn" IDENT "(" params ")" ("→" typ
params       := (param ("," param)*)?
param        := IDENT ":" type
type         := "i64" | "f64" | "bool" | "string" |
block        := "{" statement* "}"
statement    := let | assign | print | if | while |
let          := "let" "mut"? IDENT "=" expression
assign       := IDENT "=" expression
print        := "print" "(" expression ")"
if           := "if" expression block ("else" block
while        := "while" expression block
return       := "return" expression?
```

---

21. Examples

21.1 Factorial

```
fn factorial(n: i64) → i64 {
    if n ≤ 1 {
        return 1
    }
    return n * factorial(n - 1)
}

fn main() {
    print(factorial(10))
}
```

21.2 Floating point

```
fn main() {
    let x = 1.5
    let y = 2.0
    print(x + y)
    print(x * y)
}
```

21.3 Strings

```
fn main() {
    print("hello")
    print("a" + " " + "b")
}
```

21.4 Loop and sum

```
fn main() {
    let mut i = 0
    let mut sum = 0
```

```
    while i < 5 {
        sum = sum + i
        i = i + 1
    }
    print(sum)
}
```

21.5 Functions

```
fn mul(a: i64, b: i64) → i64 {
    return a * b
}

fn main() {
    print(mul(6, 7))
}
```

---

22. Troubleshooting

Program does not run

Use the LLVM backend:

```
nexus run file.nx --backend llvm
```

Link errors

Ensure NEXUS_CXX points to a clang-compatible driver.

Type errors

Run:

```
nexus check file.nx
```

Self-hosting tests fail

Ensure a clang-compatible compiler is available and NEXUS_CXX is set.

---

23. Design notes

NEXUS favors explicitness over implicit behavior. Types are stated in function signatures, variables must be declared before use, and the generated LLVM IR is readable. This makes the language suitable for learning, tooling, and systems-oriented programming.

---

24. Notes

- Functions must be declared before use.
- Use let mut for mutable variables.
- let _ = ... discards a value.
- Native backend is limited; use LLVM backend for full programs.
- Documentation is versioned with the repository.

---

25. Runtime function reference

This appendix lists the runtime functions available to generated programs.

Animation

Function
nexus_gfx_anim_create
nexus_gfx_anim_destroy
nexus_gfx_anim_draw
nexus_gfx_anim_frame

Function
nexus_gfx_anim_update

Audio

Function
nexus_gfx_music_load
nexus_gfx_music_play
nexus_gfx_music_stop
nexus_gfx_music_volume
nexus_gfx_sound_load
nexus_gfx_sound_play
nexus_gfx_sound_playing
nexus_gfx_sound_stop
nexus_gfx_sound_tone
nexus_gfx_sound_unload
nexus_gfx_sound_volume

Canvas

Function
nexus_gfx_canvas_brush
nexus_gfx_canvas_checkpoint
nexus_gfx_canvas_clear
nexus_gfx_canvas_create
nexus_gfx_canvas_destroy
nexus_gfx_canvas_draw
nexus_gfx_canvas_fill

Function
nexus_gfx_canvas_height
nexus_gfx_canvas_line
nexus_gfx_canvas_load_bmp
nexus_gfx_canvas_rect
nexus_gfx_canvas_redo
nexus_gfx_canvas_save_bmp
nexus_gfx_canvas_undo
nexus_gfx_canvas_width

Crypto

Function
nexus_sha256

Database

Function
nexus_sqlite_close
nexus_sqlite_exec
nexus_sqlite_open

ECS

Function
nexus_ecs_alive
nexus_ecs_collides
nexus_ecs_count
nexus_ecs_create

Function
nexus_ecs_destroy
nexus_ecs_draw
nexus_ecs_set_color
nexus_ecs_set_position
nexus_ecs_set_size
nexus_ecs_set_texture
nexus_ecs_set_velocity
nexus_ecs_update
nexus_ecs_x
nexus_ecs_y

Files

Function
nexus_file_exists
nexus_file_read
nexus_file_write

Graphics 2D

Function
nexus_gfx_audio_available
nexus_gfx_audio_error
nexus_gfx_backend
nexus_gfx_begin
nexus_gfx_camera_reset
nexus_gfx_camera_set

Function
nexus_gfx_circle
nexus_gfx_clear
nexus_gfx_cursor_visible
nexus_gfx_dt
nexus_gfx_end
nexus_gfx_error
nexus_gfx_fullscreen
nexus_gfx_height
nexus_gfx_image_type
nexus_gfx_init
nexus_gfx_is_fullscreen
nexus_gfx_line
nexus_gfx_maximize
nexus_gfx_poll
nexus_gfx_rect
nexus_gfx_restore
nexus_gfx_set_title
nexus_gfx_set_ui_resolution
nexus_gfx_should_close
nexus_gfx_shutdown
nexus_gfx_vsync
nexus_gfx_width

Graphics 3D

Function
nexus_gfx3d_begin
nexus_gfx3d_cube
nexus_gfx3d_end
nexus_gfx3d_grid
nexus_gfx3d_model_draw
nexus_gfx3d_model_load
nexus_gfx3d_model_unload

Input

Function
nexus_gfx_event_is
nexus_gfx_event_key
nexus_gfx_event_mouse_button
nexus_gfx_event_type
nexus_gfx_event_x
nexus_gfx_event_y
nexus_gfx_key_down
nexus_gfx_mouse_down
nexus_gfx_mouse_x
nexus_gfx_mouse_y
nexus_input
nexus_input_f64
nexus_input_i64

JSON

Function
nexus_json_get

Lists

Function
nexus_ilist_get
nexus_ilist_len
nexus_ilist_new
nexus_ilist_push
nexus_ilist_set
nexus_slist_get
nexus_slist_len
nexus_slist_new
nexus_slist_push

Math

Function
nexus_sqrt
nexus_to_f64
nexus_to_i64

Memory

Function
nexus_mem_alloc
nexus_mem_copy

Function
nexus_mem_free
nexus_mem_read
nexus_mem_set

Networking

Function
nexus_http_get

OS arguments

Function
nexus_os_args_count
nexus_os_args_get

Particles

Function
nexus_gfx_particle_alive
nexus_gfx_particle_create
nexus_gfx_particle_destroy
nexus_gfx_particles_clear
nexus_gfx_particles_draw
nexus_gfx_particles_update

Physics

Function
nexus_physics_aabb

Function
nexus_physics_circle

Printing

Function
nexus_print_bool
nexus_print_f64
nexus_print_i64
nexus_print_str

Safety

Function
nexus_bounds_check

Scenes

Function
nexus_scene_add
nexus_scene_clear
nexus_scene_create
nexus_scene_destroy
nexus_scene_load
nexus_scene_save

Screen

Function
nexus_screen_begin

Function
nexus_screen_clear
nexus_screen_end
nexus_screen_height
nexus_screen_present
nexus_screen_put
nexus_screen_set_title
nexus_screen_width

String conversion

Function
nexus_str_bool
nexus_str_concat
nexus_str_equal
nexus_str_f64
nexus_str_i64

Strings

Function
nexus_string_byte_at
nexus_string_contains
nexus_string_find
nexus_string_len
nexus_string_lower
nexus_string_slice
nexus_string_split_at

Function
nexus_string_split_count
nexus_string_trim
nexus_string_upper

System

Function
nexus_beep
nexus_capture_output
nexus_clear
nexus_env
nexus_exit
nexus_key_pressed
nexus_random_i64
nexus_read_key
nexus_sleep_ms
nexus_system
nexus_time_ms

Text and fonts

Function
nexus_gfx_font_default
nexus_gfx_font_load
nexus_gfx_font_style
nexus_gfx_font_unload
nexus_gfx_text

Function
nexus_gfx_text_font
nexus_gfx_text_height
nexus_gfx_text_width
nexus_gfx_texture_draw
nexus_gfx_texture_draw_frame
nexus_gfx_texture_filter
nexus_gfx_texture_height
nexus_gfx_texture_load
nexus_gfx_texture_unload
nexus_gfx_texture_width

UI

Function
nexus_ui_button
nexus_ui_checkbox
nexus_ui_label
nexus_ui_panel
nexus_ui_progress
nexus_ui_slider

---

25. Runtime function reference

This appendix lists the runtime functions available to generated programs.

Animation

Function
nexus_gfx_anim_create
nexus_gfx_anim_destroy
nexus_gfx_anim_draw
nexus_gfx_anim_frame
nexus_gfx_anim_update

Audio

Function
nexus_gfx_music_load
nexus_gfx_music_play
nexus_gfx_music_stop
nexus_gfx_music_volume
nexus_gfx_sound_load
nexus_gfx_sound_play
nexus_gfx_sound_playing
nexus_gfx_sound_stop
nexus_gfx_sound_tone
nexus_gfx_sound_unload
nexus_gfx_sound_volume

Canvas

Function
nexus_gfx_canvas_brush

Function
nexus_gfx_canvas_checkpoint
nexus_gfx_canvas_clear
nexus_gfx_canvas_create
nexus_gfx_canvas_destroy
nexus_gfx_canvas_draw
nexus_gfx_canvas_fill
nexus_gfx_canvas_height
nexus_gfx_canvas_line
nexus_gfx_canvas_load_bmp
nexus_gfx_canvas_rect
nexus_gfx_canvas_redo
nexus_gfx_canvas_save_bmp
nexus_gfx_canvas_undo
nexus_gfx_canvas_width

Crypto

Function
nexus_sha256

Database

Function
nexus_sqlite_close
nexus_sqlite_exec
nexus_sqlite_open

ECS

Function
nexus_ecs_alive
nexus_ecs_collides
nexus_ecs_count
nexus_ecs_create
nexus_ecs_destroy
nexus_ecs_draw
nexus_ecs_set_color
nexus_ecs_set_position
nexus_ecs_set_size
nexus_ecs_set_texture
nexus_ecs_set_velocity
nexus_ecs_update
nexus_ecs_x
nexus_ecs_y

Files

Function
nexus_file_exists
nexus_file_read
nexus_file_write

Graphics 2D

Function
nexus_gfx_audio_available
nexus_gfx_audio_error
nexus_gfx_backend
nexus_gfx_begin
nexus_gfx_camera_reset
nexus_gfx_camera_set
nexus_gfx_circle
nexus_gfx_clear
nexus_gfx_cursor_visible
nexus_gfx_dt
nexus_gfx_end
nexus_gfx_error
nexus_gfx_fullscreen
nexus_gfx_height
nexus_gfx_image_type
nexus_gfx_init
nexus_gfx_is_fullscreen
nexus_gfx_line
nexus_gfx_maximize
nexus_gfx_poll
nexus_gfx_rect
nexus_gfx_restore
nexus_gfx_set_title

Function
nexus_gfx_set_ui_resolution
nexus_gfx_should_close
nexus_gfx_shutdown
nexus_gfx_vsync
nexus_gfx_width

Graphics 3D

Function
nexus_gfx3d_begin
nexus_gfx3d_cube
nexus_gfx3d_end
nexus_gfx3d_grid
nexus_gfx3d_model_draw
nexus_gfx3d_model_load
nexus_gfx3d_model_unload

Input

Function
nexus_gfx_event_is
nexus_gfx_event_key
nexus_gfx_event_mouse_button
nexus_gfx_event_type
nexus_gfx_event_x
nexus_gfx_event_y
nexus_gfx_key_down

Function
nexus_gfx_mouse_down
nexus_gfx_mouse_x
nexus_gfx_mouse_y
nexus_input
nexus_input_f64
nexus_input_i64

JSON

Function
nexus_json_get

Lists

Function
nexus_ilist_get
nexus_ilist_len
nexus_ilist_new
nexus_ilist_push
nexus_ilist_set
nexus_slist_get
nexus_slist_len
nexus_slist_new
nexus_slist_push

Math

Function
nexus_sqrt
nexus_to_f64
nexus_to_i64

Memory

Function
nexus_mem_alloc
nexus_mem_copy
nexus_mem_free
nexus_mem_read
nexus_mem_set

Networking

Function
nexus_http_get

OS arguments

Function
nexus_os_args_count
nexus_os_args_get

Particles

Function
nexus_gfx_particle_alive

Function
nexus_gfx_particle_create
nexus_gfx_particle_destroy
nexus_gfx_particles_clear
nexus_gfx_particles_draw
nexus_gfx_particles_update

Physics

Function
nexus_physics_aabb
nexus_physics_circle

Printing

Function
nexus_print_bool
nexus_print_f64
nexus_print_i64
nexus_print_str

Safety

Function
nexus_bounds_check

Scenes

Function
nexus_scene_add

Function
nexus_scene_clear
nexus_scene_create
nexus_scene_destroy
nexus_scene_load
nexus_scene_save

Screen

Function
nexus_screen_begin
nexus_screen_clear
nexus_screen_end
nexus_screen_height
nexus_screen_present
nexus_screen_put
nexus_screen_set_title
nexus_screen_width

String conversion

Function
nexus_str_bool
nexus_str_concat
nexus_str_equal
nexus_str_f64
nexus_str_i64

Strings

Function
nexus_string_byte_at
nexus_string_contains
nexus_string_find
nexus_string_len
nexus_string_lower
nexus_string_slice
nexus_string_split_at
nexus_string_split_count
nexus_string_trim
nexus_string_upper

System

Function
nexus_beep
nexus_capture_output
nexus_clear
nexus_env
nexus_exit
nexus_key_pressed
nexus_random_i64
nexus_read_key
nexus_sleep_ms
nexus_system

Function
nexus_time_ms

Text and fonts

Function
nexus_gfx_font_default
nexus_gfx_font_load
nexus_gfx_font_style
nexus_gfx_font_unload
nexus_gfx_text
nexus_gfx_text_font
nexus_gfx_text_height
nexus_gfx_text_width
nexus_gfx_texture_draw
nexus_gfx_texture_draw_frame
nexus_gfx_texture_filter
nexus_gfx_texture_height
nexus_gfx_texture_load
nexus_gfx_texture_unload
nexus_gfx_texture_width

UI

Function
nexus_ui_button
nexus_ui_checkbox
nexus_ui_label

Function
nexus_ui_panel
nexus_ui_progress
nexus_ui_slider